

MODULE 5: Concurrency
WEEK 5

Learning Outcomes:

After completing this module you are expected to understand the following:

- 1. Dispatching and context switching
- 2. The role of interrupts
- 3. Managing atomic access to OS objects

A. Engage

Memes Station



Figure 5.0: Computer Bug

B. Explain

A **software bug** is an error, flaw or fault in a computer program or system that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. The process of finding and fixing bugs is termed "debugging" and often uses formal techniques or tools to pinpoint bugs, and since the 1950s, some computer systems have been designed to also deter, detect or auto-correct various computer bugs during operations.

C. Explore

Video Titles:

1. Context Switch

YouTube Links:

1. <https://www.youtube.com/watch?v=vTgccrbYHYs>

Module Video Filenames:

1. WEEK05_Context Switch.mp4

D. Elaborate

Introduction

1. Dispatching and context switching.

A dispatcher is a special program which comes into play after the scheduler. When the scheduler completes its job of selecting a process, it is the dispatcher which takes that process to the desired state/queue. The dispatcher is the module that gives a process control over the CPU after it has been selected by the short-term scheduler. This function involves the following:

- Switching context
- Switching to user mode
- Jumping to the proper location in the user program to restart that program

The Difference between the Scheduler and Dispatcher –

Consider a situation, where various processes are residing in the ready queue waiting to be executed. The CPU cannot execute all of these processes simultaneously, so the operating system has to choose a particular process on the basis of the scheduling algorithm used. So, this procedure of selecting a process among various processes is done by the scheduler. Once the scheduler has selected a process from the queue, the dispatcher comes into the picture, and it is the dispatcher who takes that process from the ready queue and moves it into the running state. Therefore, the scheduler gives the dispatcher an ordered list of processes which the dispatcher moves to the CPU over time.

Example –

There are 4 processes in the ready queue, P1, P2, P3, P4; Their arrival times are t0, t1, t2, t3 respectively. A First in First out (FIFO) scheduling algorithm is used. Because P1 arrived first, the scheduler will decide it is the first process that should be executed, and the dispatcher will remove P1 from the ready queue and give it to the CPU. The scheduler will then determine P2 to be the next process that should be executed, so when the dispatcher returns to the queue for a new process, it will take P2 and give it to the CPU. This continues in the same way for P3, and then P4

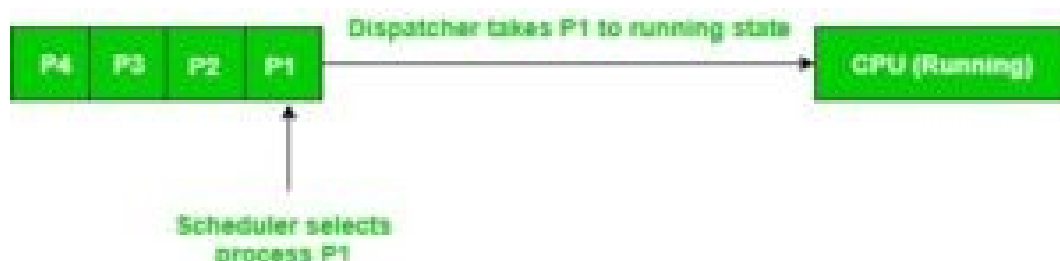


Figure 5.1. Dispatcher

A context switching is a process that involves switching of the CPU from one process or task to another. In this phenomenon, the execution of the process that is present in the running state is suspended by the kernel and another process that is present in the ready state is executed by the CPU.

It is one of the essential features of the multitasking operating system. The processes are switched so fastly that it gives an illusion to the user that all the processes are being executed at the same time.

But the context switching process involved a number of steps that need to be followed. You can't directly switch a process from the running state to the ready state. You have to save the context of that process. If you are not saving the context of any process P then after some time, when the process P comes in the CPU for execution again, then the process will start executing from starting. But in reality, it should continue from that point where it left the CPU in its previous execution. So, the context of the process should be saved before putting any other process in the running state.

A context is the contents of a CPU's registers and program counter at any point in time. Context switching can happen due to the following reasons:

- When a process of high priority comes in the ready state. In this case, the execution of the running process should be stopped and the higher priority process should be given the CPU for execution.
- When an interruption occurs then the process in the running state should be stopped and the CPU should handle the interrupt before doing something else.
- When a transition between the user mode and kernel mode is required then you have to perform the context switching.

Steps involved in Context Switching

The process of context switching involves a number of steps. The following diagram depicts the process of context switching between the two processes P1 and P2.

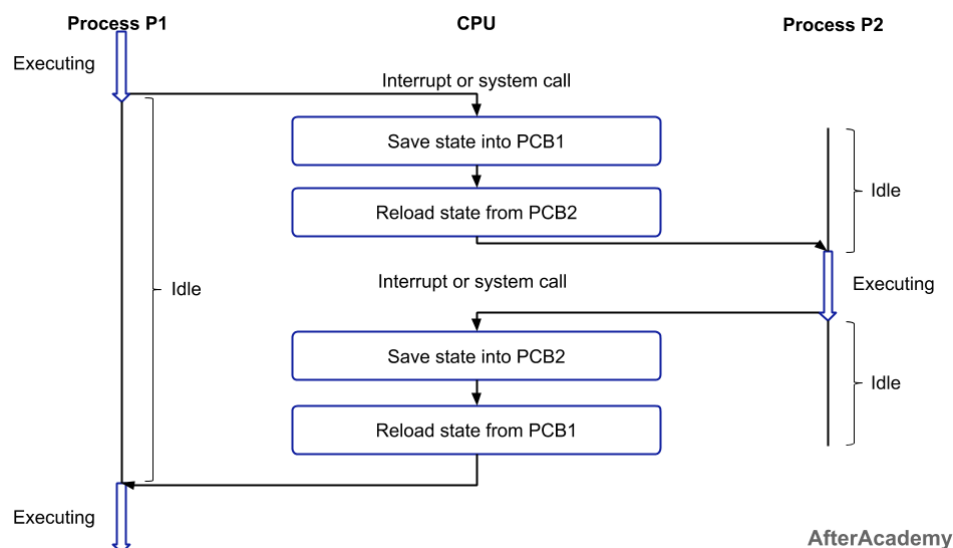


Figure 5.2. Context Switching

In the above figure, you can see that initially, the process P1 is in the running state and the process P2 is in the ready state. Now, when some interruption occurs then you have to switch the process P1 from running to the ready state after saving the context and the process P2 from ready to running state. The following steps will be performed:

1. Firstly, the context of the process P1 i.e. the process present in the running state will be saved in the Process Control Block of process P1 i.e. PCB1.

2. Now, you have to move the PCB1 to the relevant queue i.e. ready queue, I/O queue, waiting queue, etc.
3. From the ready state, select the new process that is to be executed i.e. the process P2.
4. Now, update the Process Control Block of process P2 i.e. PCB2 by setting the process state to running. If the process P2 was earlier executed by the CPU, then you can get the position of last executed instruction so that you can resume the execution of P2.
5. Similarly, if you want to execute the process P1 again, then you have to follow the same steps as mentioned above(from step 1 to 4).

For context switching to happen, two processes are at least required in general, and in the case of the round-robin algorithm, you can perform context switching with the help of one process only.

2. The role of interrupts

3. Managing atomic access to OS objects

An **interrupt** in a computing system is an event that requires stopping temporarily the progress of a process/thread (P1) in order to run a different process/thread (P2) and, usually, resume P1 once P2 is completed.

A process/thread that serves an interrupting event could itself be interrupted, which would create a cascade of interrupts that would by the end of each one return to the interrupted process/thread, as shown in Figure 1.

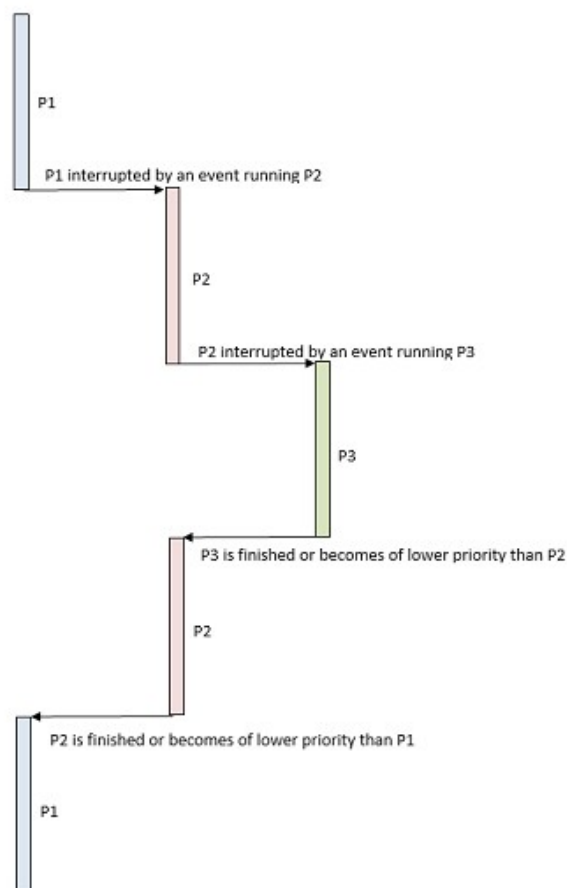


Figure 5.3. Cascaded interrupts

A very basic example of interrupts is keyboard keystrokes or mouse movement. If your cursor is focusing on a text field, ideally, whenever you strike a key on your keyboard, the computer would interrupt what it is doing and display in the text field the corresponding character.

In computing, interrupts are very important. If interrupts are not enabled or not possible, a computer would keep running one process with no way to let it know that there are higher priority tasks that need to get the attention of the processor or the operating system. In order to understand this let us consider the example of a computing system running a clock. Without interrupts, this system would never be able to do anything else as it is running a process, incrementing its time by one each second. In order to solve this issue, we would use an interrupt-enabled system that would run a lengthy process and each one second interrupt it, so that the time would be incremented by one.

Atomic Operations

In some cases, disabling interrupts might be related to a task. Let us consider the following example. Every week, you have a brief meeting with your supervisor. Because this is a very important meeting, you would define the task, which is the weekly meeting in this case, as uninterruptible, or using computer science terms, **atomic**. In other words, you are not disabling the interruption at a certain time, but you are disabling it every time the task is being carried out. We call an uninterruptible operation an atomic operation.

Often, in computer programs, including operating systems, several processes/threads might share the same variable or object. Modifying the value of a shared variable or an attribute of a shared object must be atomic. In order to understand this, let us consider a very simple example. Two threads A and B are sharing a variable $x=5$. Let us consider the following time progress:

In some cases, disabling interrupts might be related to a task. Let us consider the following example. Every week, you have a brief meeting with your supervisor. Because this is a very important meeting, you would define the task, which is the weekly meeting in this case, as uninterruptible, or using computer science terms, **atomic**. In other words, you are not disabling the interruption at a certain time, but you are disabling it every time the task is being carried out. We call an uninterruptible operation an atomic operation.

Often, in computer programs, including operating systems, several processes/threads might share the same variable or object. Modifying the value of a shared variable or an attribute of a shared object must be atomic. In order to understand this, let us consider a very simple example. Two threads A and B are sharing a variable $x=5$. Let us consider the following time progress:

Time	P1	P2	Notes
1	Read x ($x=5$)		
2		Interrupt P1 and write new value for x (for example $x=x+5=10$)	
3	Write new value for x (for example $x=x+1=6$)		The value of x , as a shared value, should be $10+1=11$, but because P1 has an obsolete local value of x , it will remain 6, which is not the correct value

In the previous example, if modifying x was an atomic operation, P2 would not have been able to modify it, until P1 is done with its modification of x .

ASIAN INSTITUTE OF COMPUTER STUDIES

Bachelor of Science in Computer Science

Course Modules

CS4114 - Operating System

4th Year – 1st Semester

Windows and Android users should be familiar with the operating system upgrade that is required every now and then. When the operating system is performing the upgrade, you will see a message telling you that you cannot power off your computer until the upgrade is over. Also, the operating system will be locked and you will not be able to do anything with your device. This way, the operating system is preventing you from any interrupt that might affect system variables. The upgrade operation is an atomic operation.

E. Evaluate

ASSESSMENT:

Instructions: You may write your answer on the Answer Sheet (AS) provided on this module.

I. TRUE OR FALSE (2-Point Each):

1. If dispatcher are not enabled or not possible, a computer would keep running one process with no way to let it know that there are higher priority tasks that need to get the attention of the processor or the operating system.
2. In computer programs, including operating systems, several processes/threads might share the same variable or object.
3. When a transition between the user mode and kernel mode is required then you have to perform the context switching.
4. A dispatcher is a special program which comes into play after the scheduler.
5. The interrupt gives the dispatcher an ordered list of processes which the dispatcher moves to the CPU over time.
6. A context switching is a process that involves switching of the CPU from one process or task to another.
7. The Context are switched so fastly that it gives an illusion to the user that all the processes are being executed at the same time.
8. A context is the contents of a CPU's registers and program counter at any point in time.
9. A very basic example of interrupts is keyboard keystrokes or mouse movement.
10. Windows and Android users should be familiar with the operating system upgrade that is required every now and then.

Laboratory Activity

Task 5.1

Instruction: Copy the soures code and paste it to your prepared ide and save it with a file named "roundrobin.cpp".

AIM: To simulate the CPU scheduling algorithm round-robin.

DESCRIPTION: To aim is to calculate the average waiting time. There will be a time slice, each process should be executed within that time-slice and if not it will go to the waiting state so first check whether the burst time is less than the time-slice. If it is less than it assign the waiting time to the sum of the total times. If it is greater than the burst-time then subtract the time slot from the actual burst time and increment it by time-slot and the loop continues until all the processes are completed.

ALGORITHM:

Step 1: Start the process

Step 2: Accept the number of processes in the ready Queue and time quantum (or) time slice

- Step 3: For each process in the ready Q, assign the process id and accept the CPU burst time
- Step 4: Calculate the no. of time slices for each process where No. of time slice for process (n) = burst time process (n)/time slice
- Step 5: If the burst time is less than the time slice then the no. of time slices =1.
- Step 6: Consider the ready queue is a circular Q, calculate
- a) Waiting time for process (n) = waiting time of process(n-1)+ burst time of process(n-1) + the time difference in getting the CPU fromprocess(n-1)
 - b) Turnaround time for process(n) = waiting time of process(n) + burst time of process(n)+ the time difference in getting CPU from process(n).
- Step 7: Calculate
- c) Average waiting time = Total waiting Time / Number of process
 - d) Average Turnaround time = Total Turnaround Time / Number ofprocess
- Step 8: Stop the process

Source Code:

```
#include<stdio.h>

int main() {
    int i,j,n,bu[10],wa[10],tat[10],t,ct[10],max; float awt=0,att=0,temp=0;

    printf("Enter the no of processes -- "); scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("\nEnter Burst Time for process %d -- ", i+1); scanf("%d",&bu[i]);
        ct[i]=bu[i];
    }
    printf("\nEnter the size of time slice -- "); scanf("%d",&t);
    max=bu[0]; for(i=1;i<n;i++) if(max<bu[i]) max=bu[i]; for(j=0;j<(max/t)+1;j++) for(i=0;i<n;i++)
    if(bu[i]!=0)
    if(bu[i]<=t) { tat[i]=temp+bu[i]; temp=temp+bu[i]; bu[i]=0;
    }
    else { bu[i]=bu[i]-t; temp=temp+t;
    }
    for(i=0;i<n;i++){ wa[i]=tat[i]-
    ct[i]; att+=tat[i]; awt+=wa[i];}
    printf("\nThe Average Turnaround time is -- %f",att/n); printf("\nThe Average Waiting time is --
    %f ",awt/n);
    printf("\n\tPROCESS\t BURST TIME \t WAITING TIME\tTURNAROUND TIME\n");
    for(i=0;i<n;i++)
    printf("\t%d \t %d \t %d \t %d \n",i+1,ct[i],wa[i],tat[i]);
    return 0;}
```

Output:

Enter the no of processes – 3
Enter Burst Time for process 1 – 24 Enter Burst Time for process 2 -- 3 Enter Burst Time for process 3 – 3
Enter the size of time slice – 3

OUTPUT:			
PROCESS	BURST TIME	WAITING TIME	TURNAROUNDTIME
1	24	6	30
2	3	4	7

3	3	7	10
---	---	---	----

The Average Turnaround time is – 15.666667 The Average Waiting time is 5.666667

References:

- 1. Explain dispatcher in operating system–
<https://bit.ly/3pMiztb>
- 2. What is Context Switching in Operating System?–
<https://afteracademy.com/blog/what-is-context-switching-in-operating-system>

Facilitated By:		
Name	:	
MS Teams Account (email)	:	
Smart Phone Number	:	